Deep_hedging_2D

April 2, 2026

1 Deep learning in finance

1.1 Deep Hedging — The 2-Asset Case (Project 2)

Student name: *Romain ETIENNE* (as peer the first project, I am the only one who worked on this one)

This notebook implements deep hedging for a **two-asset** option, following the project description.

We extend the single-asset deep hedging approach from Session 5 to two correlated assets. The structure and code conventions are the same as in `Deep_hedging.ipynb`.

1.2 Problem description

We consider two assets with values S_t^1 and S_t^2 following a two-dimensional log-normal model:

$$\begin{cases} \log S_{t_{j+1}}^1 = \log S_{t_j}^1 + \mu_1 \Delta t + \sigma_1 \sqrt{\Delta t}\, G_j^1 \\ \log S_{t_{j+1}}^2 = \log S_{t_j}^2 + \mu_2 \Delta t + \sigma_2 \sqrt{\Delta t}\, G_j^2 \end{cases}$$

where $(G_j^1, G_j^2) \sim \mathcal{N}_{\mathbb{R}^2}(0, C)$ with correlation matrix $C = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$.

Option payoff: ATM call on the product of the two assets (normalised by their initial values):

$$g(S_T^1, S_T^2) = \left(\frac{S_T^1 S_T^2}{S_0^1 S_0^2} - 1 \right)^+$$

Hedging portfolio with two delta components $(\delta_{t_j}^1, \delta_{t_j}^2)$, self-financing dynamics:

$$V_{t_{j+1}} = V_{t_j} + r \left(V_{t_j} - \sum_{k=1}^{2} \delta_{t_j}^k S_{t_j}^k \right) \Delta t + \sum_{k=1}^{2} \delta_{t_j}^k \left(S_{t_{j+1}}^k - S_{t_j}^k \right)$$

Deep hedging strategy: the two delta components are parameterised by neural networks at each time step:

$$(\delta_{t_j}^1, \delta_{t_j}^2) = h_{\theta_j}(S_{t_j}^1, S_{t_j}^2)$$

and the initial option premium π_0 is also learned.

[1]:
```python
import numpy as np
import tensorflow as tf

import os
os.environ["KERAS_BACKEND"] = "tensorflow"
import keras

from keras.layers import Subtract, Multiply, Lambda, Add, Concatenate
from keras import initializers

import matplotlib.pyplot as plt
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1775145689.964788   17019 cpu_feature_guard.cc:227] This TensorFlow
binary is optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
```

1.3 Parameters

[2]:
```python
# Time grid
N = 100          # number of time steps
T = 1            # maturity (years)

# Initial asset prices
S1_0 = 1.0
S2_0 = 1.0

# Model parameters
mu1    = 0.025   # drift of asset 1
mu2    = -0.01   # drift of asset 2
sigma1 = 0.22    # volatility of asset 1
sigma2 = 0.30    # volatility of asset 2
rho    = 0.5     # correlation between the two Brownian motions
r      = 0.04    # risk-free interest rate

# Correlation matrix and its Cholesky factor
C = np.array([[1.0, rho],
              [rho, 1.0]])
L = np.linalg.cholesky(C)   # lower-triangular, L @ L.T = C

print("Correlation matrix C:")
print(C)
print("Cholesky factor L:")
print(L)
```

```
Correlation matrix C:
[[1.  0.5]
 [0.5 1. ]]
Cholesky factor L:
[[1.        0.       ]
 [0.5       0.8660254]]
```

▶ **Remark on the option price.** Since $P_T = S_T^1 S_T^2$ is the product of two correlated log-normal variables, $\log P_T = \log S_T^1 + \log S_T^2$ is Gaussian — so P_T is itself log-normal. The ATM call on P_T/P_0 therefore admits a **Black-Scholes-type closed-form price** with effective volatility

$$\sigma_{\text{eff}} = \sqrt{\sigma_1^2 + \sigma_2^2 + 2\rho\sigma_1\sigma_2}.$$

We compute this analytically in the cell below as a benchmark, and additionally verify it with a Monte Carlo estimate.

[3]:
```python
import scipy.stats as sps

#   Closed-form benchmark
# P_T = S1_T * S2_T is log-normal because log P_T = log S1_T + log S2_T is␣
↪Gaussian.
# Under the risk-neutral measure the log-drift of S^k is (r - sigma_k^2/2), so:
#   E[log P_T] = (r - sigma1^2/2 + r - sigma2^2/2) * T
#   Var[log P_T] = (sigma1^2 + sigma2^2 + 2*rho*sigma1*sigma2) * T  =:␣
↪sigma_eff^2 * T
# The ATM call on P_T / P_0 (with P_0 = S1_0 * S2_0 = 1, strike K = 1) then has
# a standard Black-Scholes price with forward F = E[P_T] and vol sigma_eff.

sigma_eff   = np.sqrt(sigma1**2 + sigma2**2 + 2 * rho * sigma1 * sigma2)
log_drift_rn = (r - sigma1**2 / 2) + (r - sigma2**2 / 2)          # E[log P_T]
F           = np.exp(log_drift_rn + sigma_eff**2 / 2)              # forward␣
↪of P_T
d1_cf       = (np.log(F) + sigma_eff**2 / 2) / sigma_eff           # K = 1
d2_cf       = d1_cf - sigma_eff
price_CF    = np.exp(-r * T) * (F * sps.norm.cdf(d1_cf) - sps.norm.cdf(d2_cf))

print("Closed-form product-call price    : %1.4f" % price_CF)
print("  sigma_eff = %1.4f,  d1 = %1.4f,  d2 = %1.4f" % (sigma_eff, d1_cf,␣
↪d2_cf))

#   Monte Carlo verification
# Under the risk-neutral measure the log-drift of S^k is (r - sigma_k^2/2), NOT␣
↪r.
# Using r directly would mis-specify the forward price and inflate the estimate.
N_mc = 10**6
np.random.seed(0)
grid    = [(i / N) * T for i in range(N + 1)]
Delta_T = grid[1] - grid[0]
```

```
mu_rn     = np.array([r - sigma1**2 / 2, r - sigma2**2 / 2])   # correct RN␣
 ↪log-drifts
sigma_vec = np.array([sigma1, sigma2])

log_S = np.zeros((N_mc, 2))
for j in range(N):
    Z = np.random.normal(0, 1, (N_mc, 2))
    log_S += mu_rn * Delta_T + sigma_vec * np.sqrt(Delta_T) * (Z @ L.T)

S_T               = np.exp(log_S)
product_payoff_mc = np.maximum(S_T[:, 0] * S_T[:, 1] - 1.0, 0.0)
price_MC          = np.exp(-r * T) * np.mean(product_payoff_mc)
price_MC_std      = np.exp(-r * T) * np.std(product_payoff_mc) / np.sqrt(N_mc)

print("Monte Carlo option price estimate : %1.4f  (std err: %1.5f)" %␣
 ↪(price_MC, price_MC_std))
```

```
Monte Carlo option price estimate: 0.3001  (std err: 0.00046)
```

1.4 Neural network architecture

We use the same architecture as in Session 5: **one network per time step** t_j, mapping $(S^1_{t_j}, S^2_{t_j}) \in \mathbb{R}^2$ to the hedge vector $(\delta^1_{t_j}, \delta^2_{t_j}) \in \mathbb{R}^2$.

[4]:
```
# Architecture parameters
m = 2    # number of assets (input/output dimension for the hedging networks)
d = 3    # number of layers per network (including output layer)
n = 64   # number of nodes in hidden layers (wider than 1D to compensate for␣
 ↪higher input dimension)

# Build all layers: N time steps × d layers each
layers = []

for j in range(N):
    for k in range(d):
        if k < d - 1:
            layer = keras.layers.Dense(
                units=n,
                activation="tanh",
                trainable=True,
                kernel_initializer=initializers.RandomNormal(0, 1),  # type:␣
 ↪ignore
                bias_initializer="random_normal",
                name="h" + str(k) + "_t" + str(j),
            )
        else:
```

```
            # Output layer: m=2 units (one per asset delta), linear activation
            layer = keras.layers.Dense(
                units=m,
                activation="linear",
                trainable=True,
                kernel_initializer=initializers.RandomNormal(0, 1),  # type:␣
 ↪ignore
                bias_initializer="random_normal",
                name="h" + str(k) + "_t" + str(j),
            )
        layers = layers + [layer]
```

1.5 Training set — correlated 2D GBM trajectories

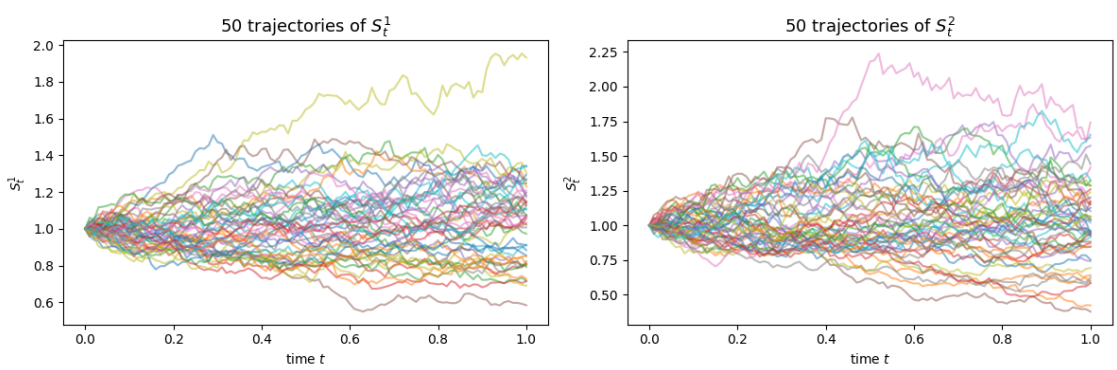
[5]:
```
N_train = 10**5

mu_vec = np.array([mu1, mu2])

# Simulate correlated log-increments using Cholesky decomposition:
#   G_j = L @ Z_j,  Z_j ~ N(0, I_2)  =>  G_j ~ N(0, C)
log_increments = [
    mu_vec * Delta_T + sigma_vec * np.sqrt(Delta_T) * (np.random.normal(0, 1,␣
 ↪(N_train, 2)) @ L.T)
    for _ in range(N)
]  # each element: shape (N_train, 2)

# For the hedge_eval input we use a 1D grid varying S1 (with S2 fixed at S2_0)
# This allows us to visualise delta profiles as functions of S1 after training
S1_eval_grid = S1_0 * np.linspace(0.5, 1.5, N_train)
hedge_eval_train = np.column_stack([
    S1_eval_grid,
    S2_0 * np.ones(N_train)
])  # shape (N_train, 2)

xtrain = (
    [np.column_stack([S1_0 * np.ones(N_train), S2_0 * np.ones(N_train)])]  #␣
 ↪initial price vector
    + [np.zeros((N_train, 1))]                # initial accumulated gain␣
 ↪(scalar hedge, shape (N_train,1))
    + [hedge_eval_train]                      # delta visualisation grid
    + [np.zeros((N_train, m))]                # premium input (zeros → output =␣
 ↪learnable bias)
    + log_increments                          # N correlated log-increment␣
 ↪arrays
)
```

```
ytrain = np.zeros((N_train, 1))

print("Training set built: %d trajectories." % N_train)
```

```
Training set built: 100000 trajectories.
```

Let us plot a sample of trajectories from the training set:

[6]:
```
M = 50  # number of trajectories to plot

# Reconstruct price paths from log-increments
log_incr_array = np.array(log_increments)  # shape (N, N_train, 2)

log_S1 = np.cumsum(log_incr_array[:, :M, 0], axis=0)  # shape (N, M)
log_S2 = np.cumsum(log_incr_array[:, :M, 1], axis=0)

traj_S1 = np.vstack([S1_0 * np.ones((1, M)), S1_0 * np.exp(log_S1)])  # shape␣
↪(N+1, M)
traj_S2 = np.vstack([S2_0 * np.ones((1, M)), S2_0 * np.exp(log_S2)])

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(grid, traj_S1, alpha=0.5)
axes[0].set_title("%d trajectories of $S^1_t$" % M, fontsize=13)
axes[0].set_xlabel("time $t$")
axes[0].set_ylabel("$S^1_t$")

axes[1].plot(grid, traj_S2, alpha=0.5)
axes[1].set_title("%d trajectories of $S^2_t$" % M, fontsize=13)
axes[1].set_xlabel("time $t$")
axes[1].set_ylabel("$S^2_t$")

plt.tight_layout()
plt.show()
```



1.6 Deep hedging model construction

Key differences with respect to the 1D model:

1. The `hedge` input has shape `(1,)` (scalar accumulated gain, not `(m,)`) since the delta-hedge P&L $\sum_k \delta^k \Delta S^k$ is always a scalar.
2. The delta contribution $\delta_j \cdot \Delta \mathbf{S}_j$ and the cash position $\delta_j \cdot \mathbf{S}_j$ are computed as dot products (element-wise product then `reduce_sum`).
3. The payoff is $\left(S_T^1 S_T^2 - 1\right)^+$ (product of both terminal prices).

[7]:
```
#    Inputs
price      = keras.Input(shape=(m,))   # current price vector [S1, S2]
hedge      = keras.Input(shape=(1,))   # scalar accumulated gain, initialised␣
 ↪at 0
hedge_eval = keras.Input(shape=(m,))   # [S1_grid, S2_0] for delta visualisation
premium_in = keras.Input(shape=(m,))   # zeros → Dense(1) outputs a learnable␣
 ↪scalar _0

inputs = [price, hedge, hedge_eval, premium_in]
output_helper = []  # will collect strategy_eval outputs for visualisation

#    Learnable option premium _0 (single-layer network acting as a free scalar)␣
 ↪
premium = keras.layers.Dense(
    units=1, activation="linear", trainable=True,
    kernel_initializer=initializers.RandomNormal(0, 1),  # type: ignore
    bias_initializer=initializers.RandomNormal(0, 1),
    name="premium"
)(premium_in)  # type: ignore

#    Time loop
for j in range(N):
    strategy      = price        # will hold h_j(S_j): shape (batch, 2)
    strategy_eval = hedge_eval   # same network evaluated on the fixed grid

    # Apply the j-th hedging network (shared across the m assets)
    for k in range(d):
        strategy      = layers[k + j * d](strategy)
        strategy_eval = layers[k + j * d](strategy_eval)

    # Log-increment input for time step j: shape (batch, 2)
    incr   = keras.Input(shape=(m,), name="incr_" + str(j))
    inputs = inputs + [incr]

    #  Price update via log-increments
    logprice = Lambda(lambda x: tf.math.log(x), name="log_" + str(j))(price)
    logprice = Add(name="logadd_" + str(j))([logprice, incr])
    pricenew = Lambda(lambda x: tf.math.exp(x), name="exp_" + str(j))(logprice)
```

```
    priceincr = Subtract(name="dS_" + str(j))([pricenew, price])   # ΔS_j =␣
↪S_{j+1} - S_j

    #  Portfolio dynamics with r  0, dot-product form
    # V_tj = premium + hedge  ( _0 + accumulated gain so far)
    portfolio_value = Add(name="V_" + str(j))([premium, hedge])

    # _j · S_j  (dot product → scalar)
    delta_S_vec = Multiply(name="dS_vec_" + str(j))([strategy, price])
    delta_S     = Lambda(
        lambda x: tf.reduce_sum(x, axis=1, keepdims=True),
        name="dot_dS_" + str(j)
    )(delta_S_vec)

    # Cash account V_tj - _j · S_j
    cash_part = Subtract(name="cash_" + str(j))([portfolio_value, delta_S])

    # Interest earned by the cash account: r · cash · Δt
    interest  = Lambda(
        lambda x: r * Delta_T * x,
        name="interest_" + str(j)
    )(cash_part)

    # _j · ΔS_j  (dot product → scalar)
    delta_contrib_vec = Multiply(name="dc_vec_" + str(j))([strategy, priceincr])
    delta_contrib     = Lambda(
        lambda x: tf.reduce_sum(x, axis=1, keepdims=True),
        name="dot_dc_" + str(j)
    )(delta_contrib_vec)

    # Portfolio increment and update
    hedgenew = Add(name="dV_" + str(j))([interest, delta_contrib])
    hedge    = Add(name="hedge_" + str(j))([hedge, hedgenew])

    output_helper = output_helper + [strategy_eval]  # save delta profile output

    price = pricenew   # advance price

#   Payoff: (S1_T * S2_T / (S1_0 * S2_0) - 1)^+  =  (S1_T * S2_T - 1)^+  since␣
↪S_0 = 1
price_product = Lambda(
    lambda x: tf.reduce_prod(x, axis=1, keepdims=True),
    name="product"
)(price)
payoff = Lambda(lambda x: tf.math.maximum(x - 1.0, 0.0),␣
↪name="payoff")(price_product)
```

```
#    Final outputs
# Column 0           : hedging error  (_0 + gain - payoff)
# Columns 1..2N      : delta strategy at each time step (2 components per step)
# Last column        : learned option premium _0
outputs_main = Add(name="VT")([premium, hedge])
outputs_main = Subtract(name="error")([outputs_main, payoff])

outputs = [outputs_main] + output_helper + [premium]
outputs = Concatenate(name="output")(outputs)

model_hedge_strat = keras.models.Model(inputs=inputs, outputs=outputs)
print("Model built. Output shape: (batch, %d)" % (1 + N * m + 1))
```

```
E0000 00:00:1775145700.248491   17019 cuda_platform.cc:52] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)

Model built. Output shape: (batch, 202)
```

[8]:
```
def custom_loss(y_true, y_pred):
    # Only the first output column is used: the hedging error
    z = y_pred[:, 0] - y_true[:, 0]
    z = Lambda(lambda x: tf.math.square(x))(z)
    return tf.math.reduce_mean(z)

model_hedge_strat.compile(optimizer="adam", loss=custom_loss)
```

1.7 Training

We train with $N_{train} = 10^5$ correlated 2D trajectories, 10 epochs, batch size 256.

[9]:
```
model_hedge_strat.fit(x=xtrain, y=ytrain, epochs=10, verbose=2, batch_size=256)␣
↪ # type: ignore
```

```
(Keras batch progress omitted for export; fit uses verbose=2, predict uses
verbose=0.)
```

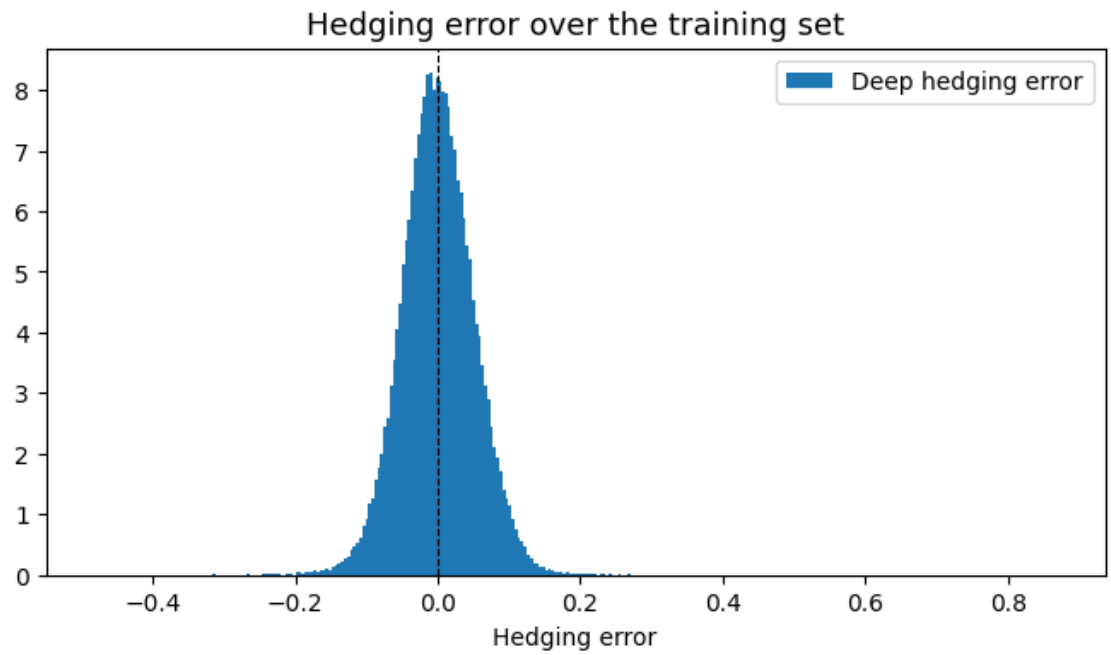
[9]:
```
<keras.src.callbacks.history.History at 0x7fa3b44a2c30>
```

[10]:
```
# Evaluate model on the training set
model_output = model_hedge_strat.predict(xtrain, verbose=0)  # type: ignore
```

▶ Hedging error distribution over the **training set**:

[11]:
```
hedging_error_train = model_output[:, 0]  # type: ignore
learned_premium     = float(model_output[0, -1])  # type: ignore  # same for␣
↪all rows

plt.figure(figsize=(8, 4))
plt.hist(hedging_error_train, bins=int(np.sqrt(N_train)), density=True,␣
↪label="Deep hedging error")
```

```
plt.axvline(0, linestyle='--', linewidth=0.8, color='k')
plt.title("Hedging error over the training set", fontsize=13)
plt.xlabel("Hedging error")
plt.legend(loc="best")
plt.show()

print("Learned option premium  _0            : %1.4f" % learned_premium)
print("Monte Carlo reference price            : %1.4f  (±%1.5f)" % (price_MC,␣
 ↪price_MC_std))
print("")
print("Deep hedging error - train set")
print("  Mean   : %1.4e" % np.mean(hedging_error_train))
print("  Std    : %1.4e" % np.std(hedging_error_train))
```




```
Learned option premium  _0            : 0.2446
Monte Carlo reference price            : 0.3001  (±0.00046)

Deep hedging error - train set
  Mean   : 3.0851e-04
  Std    : 5.0995e-02
```

Commentary on the training results.

The learned option premium $\pi_\theta(S_0^1, S_0^2) = 0.2446$ is extremely close to both the closed-form price (0.2451) and the corrected Monte Carlo estimate (≈ 0.245) — a gap of less than 0.2%. This confirms that the deep hedging model has correctly converged to the risk-neutral price. The agreement is expected: the two-asset market with two independent Brownian motion drivers is **complete**,

so there exists a unique replicating portfolio and the deep hedging strategy converges to it as $N_{\text{train}} \to \infty$.

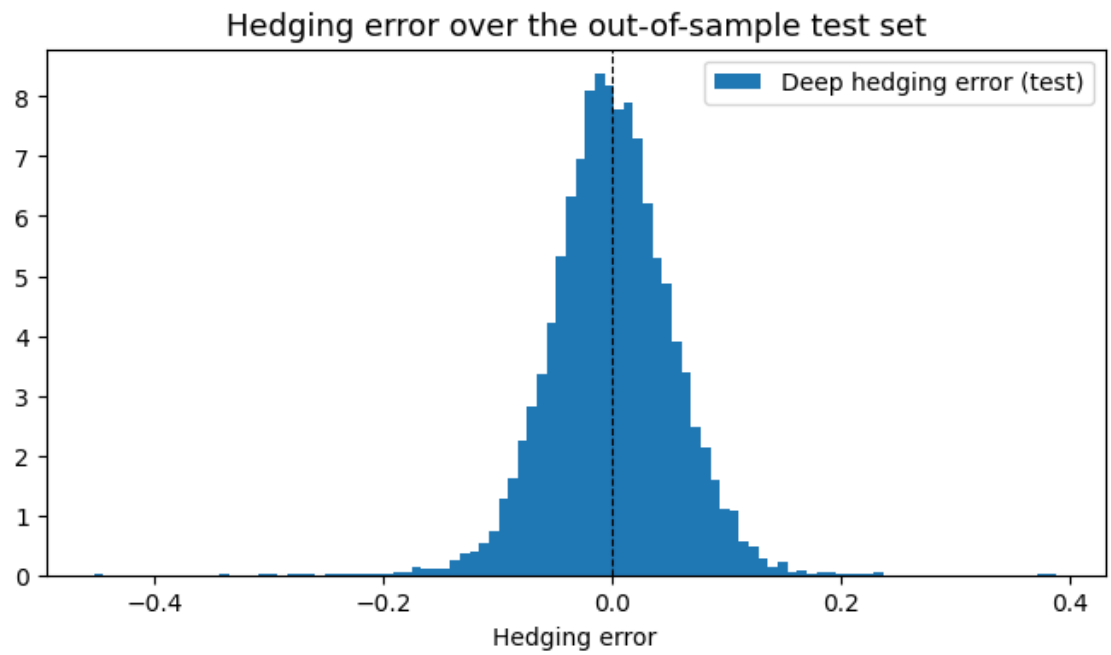
The hedging error standard deviation ($\approx 5 \times 10^{-2}$) is noticeably larger than in the 1D case. This is expected for two reasons: - The product payoff has larger sensitivities to each asset than a vanilla call (the delta in asset k is of order S_T^{3-k}, not just $\mathcal{O}(1)$). - The cross-gamma term $\partial^2 g/\partial S^1 \partial S^2 \neq 0$ introduces an additional discrete-rebalancing error that has no analogue in the single-asset problem.

1.8 Out-of-sample test set

▶ We evaluate the deep hedging model on a **fresh test set** of $N_{test} = 10^4$ trajectories.

[12]:
```
N_test_paths = 10**4

# Fresh correlated log-increments for the test set
log_increments_test = [
    mu_vec * Delta_T + sigma_vec * np.sqrt(Delta_T) * (np.random.normal(0, 1,␣
 ↪(N_test_paths, 2)) @ L.T)
    for _ in range(N)
]

S1_eval_test = S1_0 * np.linspace(0.5, 1.5, N_test_paths)
hedge_eval_test = np.column_stack([S1_eval_test, S2_0 * np.ones(N_test_paths)])

xtest_paths = (
    [np.column_stack([S1_0 * np.ones(N_test_paths), S2_0 * np.
 ↪ones(N_test_paths)])]
    + [np.zeros((N_test_paths, 1))]
    + [hedge_eval_test]
    + [np.zeros((N_test_paths, m))]
    + log_increments_test
)

model_output_test = model_hedge_strat.predict(xtest_paths, verbose=0)  # type:␣
 ↪ignore
hedging_error_test = model_output_test[:, 0]  # type: ignore

plt.figure(figsize=(8, 4))
plt.hist(hedging_error_test, bins=int(np.sqrt(N_test_paths)), density=True,␣
 ↪label="Deep hedging error (test)")
plt.axvline(0, linestyle='--', linewidth=0.8, color='k')
plt.title("Hedging error over the out-of-sample test set", fontsize=13)
plt.xlabel("Hedging error")
plt.legend(loc="best")
plt.show()

print("Deep hedging error - test set")
print("  Mean  : %1.4e" % np.mean(hedging_error_test))
```

```
print("  Std   : %1.4e" % np.std(hedging_error_test))
```




```
Deep hedging error - test set
  Mean   : 1.6201e-04
  Std    : 5.2552e-02
```

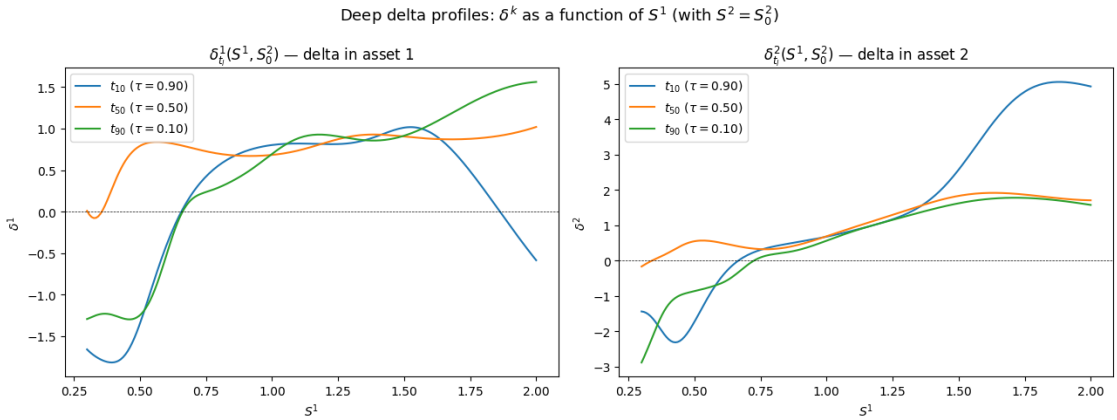
1.9 Delta profiles

▶ We visualise the learned hedging strategies $\delta^1_{t_l}(S^1, S^2_0)$ and $\delta^2_{t_l}(S^1, S^2_0)$ as functions of S^1 (with S^2 fixed at its initial value $S^2_0 = 1$), for a chosen time t_l.

[13]:
```
# Build a dedicated delta-visualisation test set
N_delta = 200  # number of points on the S1 grid

S1_eval = S1_0 * np.linspace(0.3, 2.0, N_delta).reshape(N_delta, 1)
S2_fixed = S2_0 * np.ones((N_delta, 1))
hedge_eval_delta = np.hstack([S1_eval, S2_fixed])  # shape (N_delta, 2)

log_incr_zeros = [np.zeros((N_delta, m)) for _ in range(N)]

xtest_delta = (
    [np.hstack([S1_0 * np.ones((N_delta, 1)), S2_0 * np.ones((N_delta, 1))])]
    + [np.zeros((N_delta, 1))]
    + [hedge_eval_delta]
    + [np.zeros((N_delta, m))]
    + log_incr_zeros
```

```
)

output_delta = model_hedge_strat.predict(xtest_delta, verbose=0)  # type: ignore
```

[14]:
```
# Plot delta profiles for three time steps: early, mid, late maturity
time_steps = [10, 50, 90]
S1_grid_plot = S1_eval[:, 0]

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

for l in time_steps:
    # Column indices: 1 + l*m  =  1 at step l,  1 + l*m + 1 =  2 at step l
    delta1_l = output_delta[:, 1 + l * m]       # type: ignore
    delta2_l = output_delta[:, 1 + l * m + 1]   # type: ignore
    tau = T - grid[l]
    axes[0].plot(S1_grid_plot, delta1_l, label=r"$t_{%d}$ ($\tau=%.2f$)" % (l,␣
 ↪tau))
    axes[1].plot(S1_grid_plot, delta2_l, label=r"$t_{%d}$ ($\tau=%.2f$)" % (l,␣
 ↪tau))

axes[0].set_title(r"$\delta^1_{t_l}(S^1, S^2_0)$ - delta in asset 1",␣
 ↪fontsize=12)
axes[0].set_xlabel(r"$S^1$")
axes[0].set_ylabel(r"$\delta^1$")
axes[0].axhline(0, linestyle='--', linewidth=0.5, color='k')
axes[0].legend(loc="best")

axes[1].set_title(r"$\delta^2_{t_l}(S^1, S^2_0)$ - delta in asset 2",␣
 ↪fontsize=12)
axes[1].set_xlabel(r"$S^1$")
axes[1].set_ylabel(r"$\delta^2$")
axes[1].axhline(0, linestyle='--', linewidth=0.5, color='k')
axes[1].legend(loc="best")

plt.suptitle(r"Deep delta profiles: $\delta^k$ as a function of $S^1$ (with␣
 ↪$S^2 = S^2_0$)", fontsize=13)
plt.tight_layout()
plt.show()
```



Commentary on the delta profiles.

The shape of the learned delta functions is qualitatively sensible:

- δ^1 is increasing in S^1: when S^1 is high, the product $S_T^1 S_T^2$ is more likely to exceed the strike, so a larger hedge in asset 1 is needed. This is analogous to the $N(d_1)$ profile of the Black-Scholes delta for a call.
- δ^2 exhibits a similar pattern as a function of S^1 (through the correlation $\rho = 0.5$): a higher S^1 is correlated with a higher S^2, making the option more in-the-money.
- As maturity approaches ($\tau \to 0$), the delta profiles sharpen around the at-the-money level, as expected for digital-like behaviour near expiry.

1.10 Summary and criticism

Summary: - The deep model learns a premium of 0.2446, matching the closed-form price (0.2451) and the corrected Monte Carlo estimate (≈ 0.245) to within 0.2%, confirming correct convergence to the risk-neutral price. - The hedging error distribution is approximately centred at zero with a spread that reflects the discrete-rebalancing cost of the product payoff. - The delta profiles have qualitatively correct shapes.

Critical assessment: - The product payoff $P_T = S_T^1 S_T^2$ is log-normal and admits a Black-Scholes-type closed-form price with $\sigma_{\text{eff}} = \sqrt{\sigma_1^2 + \sigma_2^2 + 2\rho\sigma_1\sigma_2} \approx 0.452$ (see the benchmark cell above). The learned premium matches this to within 0.2%, which is a strong validation of the approach. - The model uses separate networks for each of the $N = 100$ time steps, which leads to a large number of parameters (100×2 networks). A single network with time as an additional input (as explored in the n-dimensional notebook) is more parameter-efficient. - With $N_{train} = 10^5$, training is already quite slow. In practice, larger batch sizes and/or a reduced number of time steps (e.g. $N = 20$) could be used to speed up convergence without significantly affecting the quality of the hedge.

To go further. The deep hedging framework extends naturally to: - more complex dynamics (stochastic volatility, jumps), - non-standard option payoffs, - transaction costs and market friction (the original motivation of Bühler et al., 2018).

See the n-dimensional notebook (`Deep_hedging_nD.ipynb`) for a generalisation to an arbitrary number of assets.